MatLan - Language Design and Grammar

1. Introduction

- MatLan is a domain-specific language designed to simplify and express matrix operations, particularly those commonly used in LLM computations. It aims to provide a clear and concise syntax for expressing matrix algebra.

2. Language Design

- **2.1 Data Types**
  - ‘Number’: Floating-point numbers (e.g., 3.14, -2.0, 1e-5). This data type is used to represent scalar values within matrices.
  - ‘Boolean’: ‘true’, ‘false’. Used for logical operations and conditional control flow.
  - ‘Matrix’: A two-dimensional array of Number values. Matrices are the fundamental data structure for representing tensors and weight matrices in LLM models. Syntax for matrix definition: matrix[[1.0, 2.0], [3.0, 4.0]]
  - ‘String’: Sequence of characters enclosed in double quotes (e.g., "hello"). Used for output in print statements.

- **2.2 Operators**
  - Assignment: ‘=’. Assigns a value to a variable.
  - Arithmetic:
    - +: Matrix addition. Requires matrices of the same dimensions.
    - -: Matrix subtraction. Requires matrices of the same dimensions.
    - *: Matrix multiplication. The number of columns in the first matrix must equal the number of rows in the second matrix.
    - .*: Element-wise multiplication (Hadamard product). Requires matrices of the same dimensions.
    - /: Matrix division. This operation can be implemented as syntactic sugar for multiplication by the inverse of a matrix.

    - Relational:
        - >: Greater than (for scalar comparisons).
        - <: Less than (for scalar comparisons).
        - ==: Equal to (for scalar comparisons).
        - !=: Not equal to (for scalar comparisons).
        - >=: Greater than or equal to (for scalar comparisons).
        - <=: Less than or equal to (for scalar comparisons).
    - Ternary: ?:. A conditional operator that evaluates one of two expressions based on a boolean condition.
- **2.3 Control Flow**
    - if-then-else: Conditional execution of code blocks based on a boolean expression.
    - for loop: Iterative execution of a code block for a specified range.
    - while loop: Iterative execution of a code block as long as a boolean condition is true.
- **2.4 Statements**
    - print: Outputs the value of an expression to the console. Can print Number, Boolean, String, and Matrix values.

3. Grammar (EBNF)

This section formally defines the syntax of MatLan. The lexical grammar specifies the structure of individual tokens, while the syntactic grammar defines how these tokens are combined to form valid programs.
Extended Backus-Naur Form (EBNF)

- **3.1 Lexical Grammar (Tokens)**
    - Defines the basic building blocks of the language.
    - NUMBER ::= ('-')? Digit+ ('.' Digit+)? ((E|e) ('+'|'-')? Digit+)?
        - Digit ::= '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7' | '8' | '9'
    - BOOLEAN ::= 'true' | 'false'
    - IDENTIFIER ::= Letter (Letter | Digit)*
        - Letter ::= 'a' | 'b' | ... | 'z' | 'A' | 'B' | ... | 'Z'

    - MATRIX ::= 'matrix' '[' NUMBER ',' NUMBER ']' | 'matrix' '[' '[' NumberList ']' (',' '[' NumberList ']')* ']' ']'
        - NumberList ::= NUMBER (',' NUMBER)*
    - STRING ::= '"' ([^”] | '\"')* '"'
    - Operators: '+' | '-' | '*' | '.*' | '/' | '=' | '==' | '!=' | '>' | '<' | '\?' | ':'
    - Keywords: 'if' | 'then' | 'else' | 'for' | 'while' | 'print' | 'matrix' | 'or' | 'and' | 'not'
    - Other: ';' | ',' | '(' | ')' | '{' | '}' | '[' | ']'
    - Whitespace and Comments:
        - WHITESPACE ::= (' ' | '\t' | '\n')+
        - COMMENT ::= '//' .* '\n'

- **3.2 Syntactic Grammar**

The syntactic grammar defines how the tokens specified in the lexical grammar are structured to form valid MatLan programs. It uses EBNF rules to describe the hierarchical arrangement of statements, expressions, and other program constructs. For example, an assignment statement is defined as an identifier followed by the assignment operator and an expression: assignmentStmt ::= IDENTIFIER '=' expression . The following EBNF rules provide a complete specification of the MatLan syntactic grammar.

```ebnf program ::= statementList . statementList ::= statement* .
statement ::= assignmentStmt ';' | ifStmt | forStmt | whileStmt |
printStmt ';' | block .

block ::= '{' statementList '}' .
assignmentStmt ::= IDENTIFIER '=' expression .
ifStmt ::= 'if' '(' expression ')' 'then' block ('else' block)? .
forStmt ::= 'for' '(' assignmentStmt ';' expression ';' assignmentStmt
')' block .
whileStmt ::= 'while' '(' expression ')' block .
printStmt ::= 'print' expressionList .
expressionList ::= expression (',' expression)* .
expression ::=
    ternaryExpr .

ternaryExpr ::=
```

```
    logicalOrExpr '?' expression ':' ternaryExpr .

logicalOrExpr ::=
    logicalAndExpr ('or' logicalAndExpr)* .

logicalAndExpr ::=
    equalityExpr ('and' equalityExpr)* .

equalityExpr ::=
    relationalExpr (('==' | '!=') relationalExpr)* .

relationalExpr ::=
    additiveExpr (('>' | '<' | '>=' | '<=') additiveExpr)* .

additiveExpr ::=
    multiplicativeExpr (('+' | '-') multiplicativeExpr)* .

multiplicativeExpr ::=
    primaryExpr (('*' | '/' | '.*') primaryExpr)* .

primaryExpr ::=
    IDENTIFIER
    | NUMBER
    | BOOLEAN
    | MATRIX
    | STRING
    | '(' expression ')'
    | unaryOp expression .

unaryOp ::= '-' | 'not' .
```